
RAPPORT SAE IA

Semestre 5

*Analyse de visages par Intelligence Artificielle
Prédiction de l'âge, du genre et de l'ethnicité*

ÉTUDIANTS

BOUAZZA Mounir
CHERRY Delrone
HALIL Kenza
MATAICHE Selma
RACHIDI Yacine

ENCADRANTS

BILAL Faye
HANANE Azzag

1. Présentation du projet	3
2. Les données	3
a. Analyses des données représentées	3
b. Analyses des caractéristiques des images	5
c. Préparation du jeu de donnée	6
3. Les modèles	7
a. Les modèles spécialisés	7
b. Le modèle multi-tâche	16
c. Le modèle par transfert de connaissance	20
4. L'application Android	24
a. Architecture technique et gestion des modèles	24
b. Le traitement de l'image : de la caméra à l'IA	24
c. Fonctionnalités et expérience utilisateur	25

1. Présentation du projet

Dans le cadre de cette SAÉ, nous devons concevoir et développer une application mobile Android capable d'analyser un visage humain afin d'en déduire **l'âge, le genre et l'ethnicité**. L'objectif principal est **d'intégrer des modèles d'intelligence artificielle** dans cette application mobile.

Ainsi, les principaux objectifs de ce projet sont les suivants :

- Détecter un visage à partir d'une photo.
- Prédire l'âge, le genre et l'ethnicité à partir du visage.
- Développer 5 modèles différents :
 - 3 modèles spécialisés : un pour l'âge, le genre et l'ethnicité.
 - 1 modèle multi-tâche pour la prédiction simultanée.
 - 1 modèle par transfert de connaissance.
- Évaluer les performances des différents modèles à l'aide des métriques adaptées.
- Déployer le meilleur modèle dans l'application.
- Développer une interface utilisateur intuitive avec :
 - Authentification et gestion de compte.
 - Possibilité d'importer des photos à partir de la galerie du téléphone et de prendre une photo directement via la caméra.
 - Affichage des résultats.
 - Historique des prédictions.
 - Informations sur le modèle utilisé.

2. Les données

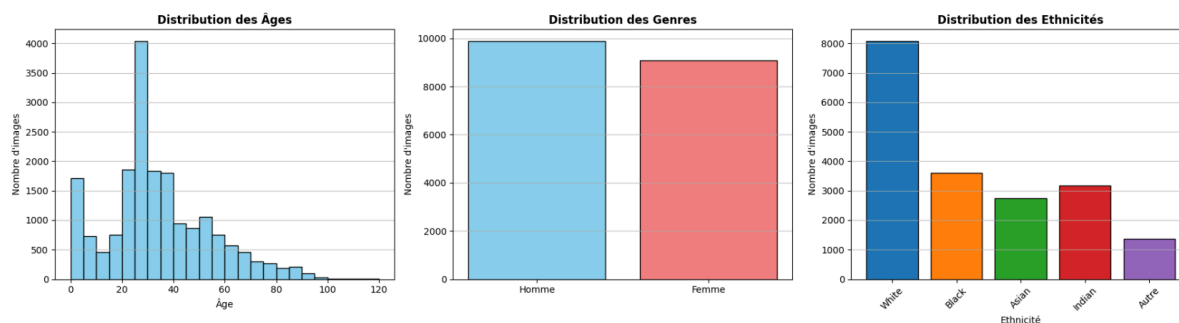
a. Analyses des données représentées

Avant toutes choses, il est essentiel de commencer par une analyse précise des données à traiter. Pour les différents modèles, nous devons utiliser le dataset UTKFace. Ainsi, nous avons procédé à une analyse de différents aspects de ce dataset. Pour cela, nous avons étudié la répartition des informations cibles (âge, genre, ethnie).

La distribution des **âges** s'étend de 1 à 116 ans. Cependant le dataset est fortement déséquilibré, avec une représentation très élevée de personnes entre 15 et 40 ans, et en particulier un pic marqué autour de la vingtaine. Les personnes âgées deviennent très rares au-delà de 60 ans. Donc le modèle sera plus performant sur des visages de jeunes et jeunes adultes mais sa capacité à généraliser correctement pour les âges extrêmes sera plus faibles (car peu d'exemple pour apprendre leur caractéristiques spécifiques).

La distribution des **genres**, quant à elle, est plutôt équilibrée, avec une légère surreprésentation des hommes par rapport aux femmes, mais cette différence reste petite et ne devrait pas introduire de biais majeur dans l'apprentissage.

La distribution des **ethnies** révèle un déséquilibre important. La catégorie *White* est très représentée, avec environ 8 000 images, soit plus du double que la plupart des autres groupes ethniques. Les catégories *Black*, *Asian* et *Indian* sont représentées de manière équivalente, tandis que la catégorie *Other* est significativement sous-représentée (mais cette catégorie représente surtout les images qui n'ont pas de labels liés à l'ethnie). Cette répartition implique que le modèle sera plus performant sur les images de catégorie *White* et surtout cela peut mener à un apprentissage biaisé.

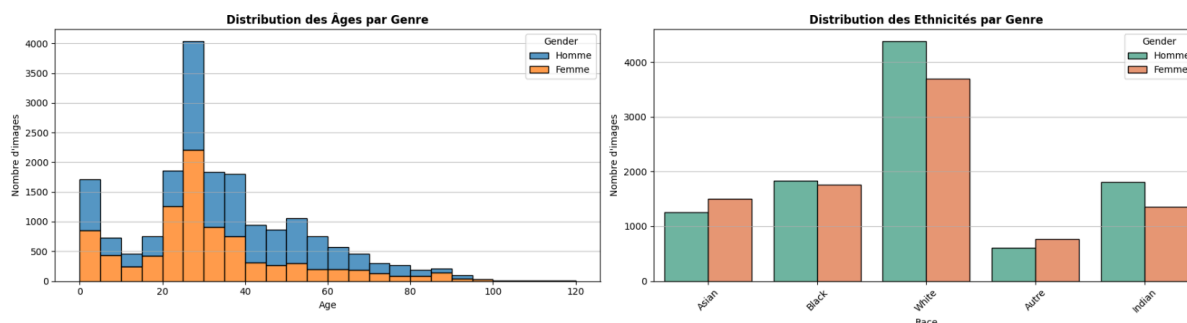


Après avoir étudié les caractéristiques isolément, nous avons étudié les distributions croisées.

La répartition des **âges par genre** montre que la répartition est similaire entre homme et femme. Cependant, on observe que les hommes sont systématiquement plus nombreux que les femmes dans presque toutes les classes d'âge, et ce déséquilibre devient plus marqué à partir de 40 ans, où les visages masculins dominent clairement les effectifs. Les femmes sont relativement mieux représentées chez les plus jeunes (enfants et adolescents). Cette différence de proportion pourrait amener le modèle à mieux généraliser sur les visages masculins, notamment chez les adultes et les seniors, et à être moins performant pour les femmes dans cette tranche d'âge moins représentées. De même, pour la tranche d'âge où les femmes sont mieux représentées.

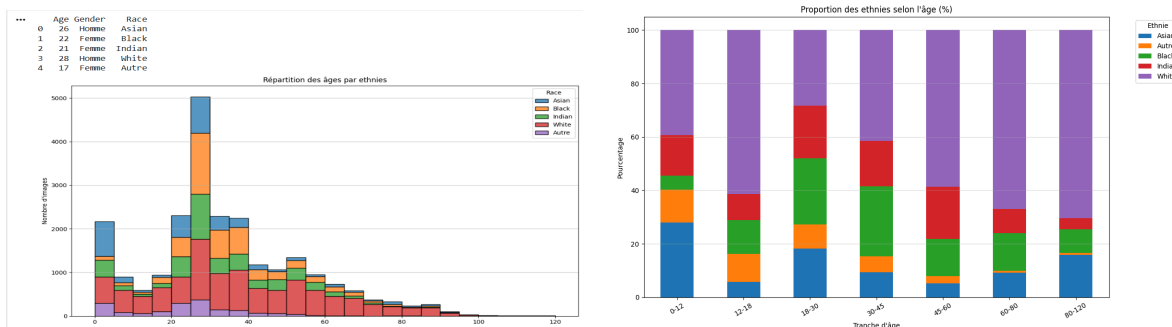
La répartition des **ethnies par genre** montre également un déséquilibre mais plus légers : pour chaque groupe ethnique, les hommes sont légèrement plus nombreux que les femmes, mais il n'y a pas d'écart trop important.

Cette homogénéité limite le risque d'un biais croisé "genre × ethnie". De manière générale, ces distributions combinées montrent que le dataset est surtout composé de visages masculins jeunes et blanc, ce qui est une limite importante à prendre en compte.



La répartition de **l'âge par l'ethnie** montre que les ethnies ne couvrent pas les âges de la même manière. Au-delà du fait que la catégorie *White* est plus représentée de manière générale, la catégorie est bien représentée sur toute la plage d'âge. Donc, le modèle pourra apprendre une progression d'âge complète pour cette ethnie. Or, le modèle peut aussi apprendre un faux lien : les

personnes âgées, appartiennent à la catégorie *White* au lieu d'apprendre l'âge réel (car plus de visage blanc représenté vers ces âges). Les ethnies *Black*, *Asian*, et *Indian* sont principalement représentées entre 20 et 40 ans. Le modèle ne verra pas forcément le vieillissement pour ces groupes et risque d'apprendre des liens ethniques plutôt que des caractéristiques visuelles du vieillissement, ce qui représente un biais important dans la prédiction de l'âge. Les prédictions seront plus fiables pour les visages blancs et moins robustes pour les autres catégories d'ethnie, notamment aux âges extrêmes.



b. Analyses des caractéristiques des images

Après avoir effectué l'analyse des données représentées, nous avons effectué une analyse des caractéristiques des images.

La **résolution** de toutes les images est de 200×200, mais nous avons décidé de changer la résolution vers 128×128 pour de meilleures performances et utiliser moins de ressources. En effet, dans un réseau CNN le temps de calcul vient principalement de la couche de convolution. Le nombre d'opérations de la couche peut être calculé avec :

$$H \times W \times K^2 \times \text{Cin} \times \text{Cout}$$

H, W = hauteur et largeur de l'image

K = taille du noyau de convolution

Cin = nombre de canaux d'entrée

Cout = nombre de filtres

Ainsi, la complexité dépend directement de la taille de l'image, et réduire la résolution de l'image réduira aussi le nombre de calculs.

De plus, les tâches de prédiction d'âge, de genre et d'ethnie nécessitent des caractéristiques du visage globales, alors, une résolution plus élevée augmente le bruit et le risque de sur-apprentissage sans améliorer significativement la performance.

Les images sont représentées en **couleurs** pour 23 097 d'entre elles et 611 sont en noir et blanc. Mais nous avons observé qu'elles sont tout de même toutes représentées sur **trois canaux RGB**.

Nous avons décidé de garder toutes les images sur 3 canaux, car même si réduire l'entrée du réseau de neurone de trois canaux à un canal (si on avait décidé de convertir en niveau de gris), aurait diminué le nombre de paramètres de la première couche de convolution, cette réduction reste minime par rapport au coût total du réseau. De plus, conserver les images sur trois canaux permet de conserver les informations de colorimétrie qui peuvent être pertinentes pour la tâche de prédiction d'ethnie notamment. Une conversion en niveaux de gris supprimerait ces informations et forcerait le

réseau à ne s'appuyer que sur les variations d'intensité lumineuse. De plus, les architectures de transfert d'apprentissage (MobileNet, EfficientNet, ResNet) sont pré-entraînées sur ImageNet avec des images RGB. Modifier le nombre de canaux d'entrée impliquerait soit de réentraîner la première couche, soit d'adapter les poids pré-entraînés, ce qui réduirait l'intérêt du transfert de connaissances.

c. Préparation du jeu de donnée

Pour chaque modèle nous devons préparer les données (les images) pour les rendre compatibles avec les formats attendus en entrée de nos modèles. En effet, les images doivent être converties en tenseurs. Chaque image doit avoir la même taille, nous avons choisis de les mettre à la taille 128×128. Pour certains modèles nous transformons les images en niveau de gris sur 1 canal car passer de 3 canaux à 1 canal permet de diminuer les calculs, pour les tâches où la couleur n'est pas utile. Ensuite, on normalise les valeurs des pixels en les divisant par 255 pour les ramener dans l'intervalle [0,1]. Cette étape de normalisation permet d'éviter au modèle de manipuler des valeurs trop grandes ce qui ralentit la convergence du réseau.

```
X = images.reshape(images.shape[0], 128, 128, 1)
X = X.astype('float32') / 255.0
```

Enfin, on forme les labels en fonction de la tâche : on utilise des valeurs continues pour la régression (pour déterminer l'âge), des valeurs binaires pour les classification à deux classes (0 ou 1 pour déterminer le genre), et des entiers (pour déterminer les ethnies avec un entier pour chaque classe). Cette préparation permet de donner au modèle des données qu'il "sait" exploiter et d'obtenir un apprentissage de meilleure qualité.

```
y_age = labels[:, 0].astype(np.float32)      # régression
y_gender = labels[:, 1].astype(np.float32)    # binaire
y_ethnicity = labels[:, 2].astype(np.int32)   # multi-classes
```

3. Les modèles

a. Les modèles spécialisés

Dans cette partie, nous expliquons comment nous avons fait évoluer nos architectures, en partant des modèles simples pour arriver à des modèles plus robustes et performants.

Modèle de classification du genre

1. Introduction

La classification du genre à partir d'images faciales est un problème de classification binaire en vision par ordinateur. L'objectif est de déterminer, à partir d'une photographie de visage, si le sujet est un homme ou une femme. Ce type de modèle est utilisé dans l'adaptation d'interfaces, l'analyse démographique ou encore les systèmes de recommandation personnalisés.

Le modèle développé est un réseau de neurones convolutif (CNN) entraîné intégralement from scratch, conçu pour recevoir des images en niveaux de gris et produire une probabilité binaire en sortie.

2. Architecture du modèle

Le réseau est structuré en quatre blocs convolutionnels avec une profondeur de filtres croissante (32, 64, 128, 128). Chaque bloc est composé d'une couche de convolution avec activation ReLU et padding identique, suivie d'une normalisation par batch et d'un Max Pooling. Les deux premiers blocs élargissent progressivement le champ de filtres tandis que les deux derniers maintiennent une profondeur constante, suffisante pour capturer les caractéristiques discriminantes du genre (forme du visage, structure des traits).

Après l'extraction des caractéristiques, un aplatissement (Flatten) transforme les cartes spatiales en un vecteur unidimensionnel. Deux couches entièrement connectées avec régularisation L2 assurent la classification, avec une normalisation par batch et un Dropout entre les deux pour limiter le surapprentissage. La couche de sortie est un unique neurone avec activation sigmoïde, produisant une probabilité entre 0 (homme) et 1 (femme).

3. Prétraitement des données

Les images sont converties en niveaux de gris et redimensionnées à 128×128 pixels (un seul canal). Le choix du niveau de gris repose sur le constat que la classification du genre s'appuie principalement sur la morphologie faciale plutôt que sur la couleur de peau ou les teintes. Ce choix réduit également de façon significative la taille du modèle, rendant celui-ci compatible avec un déploiement sur appareil mobile.

La normalisation des pixels est réalisée par une simple division par 255, ramenant les valeurs dans l'intervalle [0, 1]. Cette normalisation directe est compatible avec les contraintes d'inférence sur des environnements mobiles (TFLite). Le découpage des données est réalisé de manière stratifiée afin de conserver la proportion hommes/femmes dans chaque sous-ensemble.

4. Entraînement

Le modèle est entraîné avec la fonction de perte d'entropie croisée binaire (binary crossentropy), adaptée aux problèmes de classification à deux classes. L'optimiseur Adam est utilisé pour sa convergence rapide et son adaptation automatique du taux d'apprentissage par paramètre.

Comme pour le modèle d'âge, deux mécanismes de régulation accompagnent l'entraînement : un arrêt anticipé (Early Stopping) qui surveille la perte de validation et restaure les meilleurs poids observés, et un planificateur de taux d'apprentissage qui réduit progressivement le pas d'optimisation en cas de stagnation. Ces stratégies permettent d'obtenir un modèle bien généralisé tout en évitant le surapprentissage.

Une augmentation de données géométrique légère (retournement horizontal et rotation de faible amplitude) est appliquée pendant l'entraînement afin d'améliorer la robustesse du modèle face à la variabilité des poses.

5. Résultats et interprétation

Le modèle atteint de très bonnes performances sur l'ensemble de test, avec une accuracy élevée et des métriques de précision et de rappel équilibrées entre les deux classes. La matrice de confusion confirme que les erreurs de classification se répartissent de manière symétrique, sans biais notable en faveur de l'une ou l'autre catégorie.

Les métriques par classe (précision, rappel, F1-score) sont proches et satisfaisantes pour les deux genres. Cet équilibre traduit le fait que le jeu de données présente une répartition relativement homogène entre hommes et femmes, et que le modèle a su exploiter les différences morphologiques faciales de manière efficace.

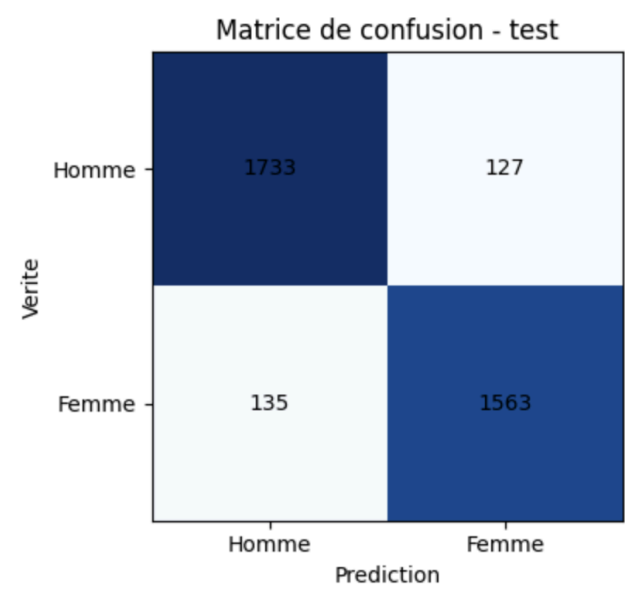
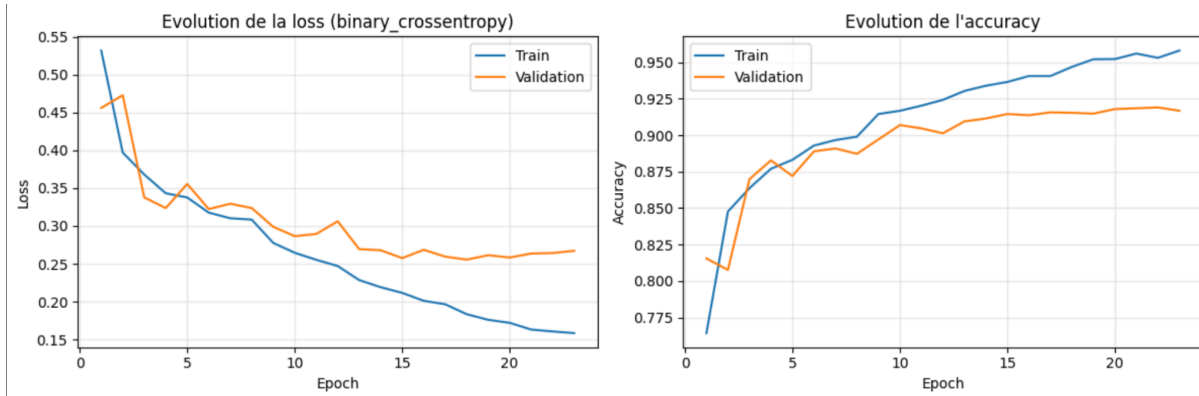
Globalement, le CNN parvient à capturer les caractéristiques discriminantes du genre même en niveaux de gris, confirmant que la forme du visage et la structure des traits sont des indices suffisants pour cette tâche de classification binaire.

6. Limites

Malgré ses bonnes performances, le modèle présente certaines limites. La conversion en niveaux de gris, bien que bénéfique pour la légèreté du modèle, élimine des informations potentiellement utiles dans certains cas ambigus (couleur des lèvres, présence de maquillage). Certaines erreurs de classification pourraient être liées à cette perte d'information.

Par ailleurs, la notion de genre telle qu'apprise par le modèle repose exclusivement sur l'apparence faciale. Les cas de visages ambigus, partiellement occultés ou présentant des expressions inhabituelles peuvent induire des erreurs. Le modèle ne tient pas compte de facteurs externes comme la coiffure, les accessoires ou le contexte, qui pourraient affiner la prédiction.

Enfin, la qualité et la diversité des images d'entraînement conditionnent directement la capacité de généralisation. Un jeu de données présentant un biais ethnique ou générationnel pourrait se traduire par des performances inégales selon les populations, ce qui constitue une limite à prendre en compte lors du déploiement en conditions réelles.



Modèle pour l'âge

La prédiction de l'âge à partir d'images faciales constitue un problème de régression en vision par ordinateur. L'objectif est d'estimer, à partir d'une photographie de visage, l'âge réel du sujet en années. Ce type de modèle trouve des applications concrètes dans le contrôle d'accès par tranche d'âge, l'adaptation d'interfaces utilisateur ou encore l'analyse démographique.

Le modèle développé dans ce cadre est un réseau de neurones convolutif (CNN) entraîné intégralement from scratch, sans recours à des poids pré-entraînés. Le but est de produire une valeur numérique continue représentant l'âge estimé.

2. Architecture du modèle

Le CNN s'organise en quatre blocs convolutionnels successifs dont la profondeur de filtres croît progressivement (32, 64, 128, 256). Chaque bloc comprend une couche de convolution avec activation ReLU et padding identique, suivie d'une normalisation par batch (Batch Normalization) et d'un sous-échantillonnage par Max Pooling. Cette progression permet au réseau d'extraire des caractéristiques de plus en plus abstraites : textures fines dans les premières couches, puis structures faciales globales dans les couches profondes.

Après le dernier bloc convolutionnel, un Global Average Pooling agrège les cartes de caractéristiques en un vecteur compact, limitant ainsi le nombre de paramètres et le risque de surapprentissage. Deux couches entièrement connectées (Dense) avec régularisation L2 et Batch Normalization assurent la projection vers l'espace de prédiction. Un Dropout est appliqué entre ces couches pour renforcer la généralisation. La sortie finale est une unique neurone sans fonction d'activation, conformément au cadre de régression linéaire.

3. Prétraitement des données

Les images sont redimensionnées à une résolution fixe de 128×128 pixels en couleur (trois canaux RGB). Cette taille offre un compromis entre la quantité d'informations visuelles conservées et le coût de calcul lors de l'entraînement.

Une standardisation statistique est appliquée sur les pixels : la moyenne et l'écart-type globaux sont calculés exclusivement sur l'ensemble d'entraînement, puis utilisés pour centrer et réduire l'ensemble des images (entraînement, validation et test). Ce choix garantit qu'aucune information issue des données de test ne fuit dans la normalisation. Les labels d'âge subissent également une normalisation (centrage-réduction) afin de faciliter la convergence de l'optimisation sur la fonction de perte.

4. Entraînement

L'entraînement repose sur la minimisation de l'erreur quadratique moyenne (MSE), fonction de perte classique pour les problèmes de régression. L'optimiseur Adam est utilisé pour sa capacité à adapter automatiquement le taux d'apprentissage pour chaque paramètre, ce qui accélère la convergence.

Deux mécanismes de régulation sont mis en place. D'une part, un arrêt anticipé (Early Stopping) surveille la perte de validation et interrompt l'entraînement lorsque celle-ci stagne, en restaurant les poids du meilleur état observé. D'autre part, un planificateur réduit progressivement le taux d'apprentissage lorsque aucune amélioration n'est constatée sur plusieurs époques consécutives. Ces

deux stratégies combinées permettent d'éviter le surapprentissage tout en obtenant une convergence fine.

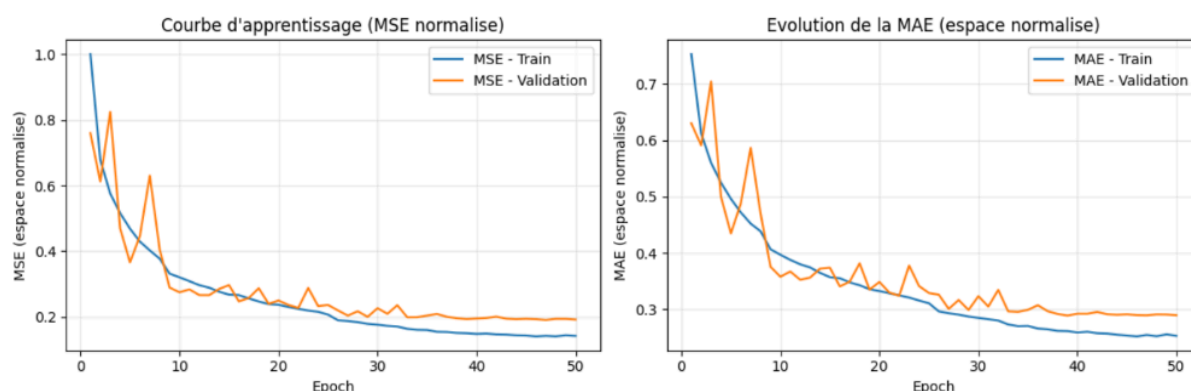
Une augmentation de données légère est également appliquée lors de l'entraînement : retournement horizontal aléatoire et rotation de faible amplitude. Ces transformations purement géométriques enrichissent la variabilité des données sans altérer la distribution statistique des pixels.

5. Résultats et interprétation

Le modèle atteint une erreur absolue moyenne (MAE) de 5.87 ans sur l'ensemble de test, ce qui témoigne de sa capacité à estimer l'âge dans la majorité des cas. Toutefois, une analyse plus fine par tranche d'âge révèle des disparités significatives de performance.

Les âges intermédiaires (approximativement entre 20 et 45 ans) sont prédits avec une grande précision. Ces tranches, largement majoritaires dans le jeu de données, fournissent suffisamment d'exemples pour que le réseau apprenne les caractéristiques faciales discriminantes associées.

En revanche, les performances se dégradent nettement pour les bébés et les très jeunes enfants (0–5 ans), où l'erreur est considérablement plus élevée. Les personnes âgées (au-delà de 60–70 ans) présentent également des erreurs accrues, bien que dans une moindre mesure. Ce phénomène s'explique principalement par la sous-représentation de ces classes extrêmes dans le jeu de données. Le modèle, exposé à un nombre insuffisant d'exemples pour ces tranches, tend à ramener ses prédictions vers la zone centrale de la distribution des âges.



6. Tentatives d'amélioration non concluantes

Plusieurs stratégies ont été expérimentées afin d'améliorer les performances du modèle sur les tranches d'âge sous-représentées. Aucune d'entre elles n'a produit de résultats stables ou concluants.

Duplication d'images pour les classes rares

Les images appartenant aux tranches d'âge extrêmes (bébés, personnes âgées) ont été dupliquées afin de rééquilibrer artificiellement la distribution des données. Cette approche a produit des résultats instables : le modèle avait tendance à surapprendre sur les exemples dupliqués sans réellement améliorer sa capacité de généralisation. Les performances globales ont été perturbées, avec des oscillations importantes de la perte de validation.

Data augmentation ciblée

Des transformations géométriques spécifiques (rotations, symétries, translations) ont été appliquées de manière ciblée sur les images des classes rares. Malgré l'augmentation du volume de données pour ces catégories, le modèle n'a pas montré d'amélioration significative. Les échantillons augmentés, trop proches des originaux, n'apportaient pas de diversité suffisante pour enrichir réellement l'apprentissage. Le risque de surapprentissage sur ces variations artificielles restait élevé.

Modification de la luminosité et du contraste

Des variations aléatoires de luminosité et de contraste ont été introduites sur les images des classes sous-représentées. Cette stratégie a altéré la distribution globale des pixels, rendant l'apprentissage incohérent avec la standardisation appliquée. Les résultats obtenus étaient dégradés par rapport au modèle de référence, suggérant que ces perturbations introduisent un biais artificiel nuisant à la capacité du réseau à généraliser.

Pondération de la fonction de perte

Un système de pondération a été mis en place afin d'attribuer un poids plus élevé aux erreurs commises sur les classes rares dans le calcul de la fonction de perte. Bien que cette idée soit théoriquement pertinente, elle a entraîné des résultats déséquilibrés : l'amélioration marginale sur les âges extrêmes s'accompagnait d'une dégradation notable sur les âges intermédiaires, classes pourtant déjà bien prédites. Le modèle perdait en stabilité globale sans parvenir à corriger efficacement les erreurs visées.

7. Limites

La principale limite du modèle réside dans le déséquilibre du jeu de données. La forte sous-représentation des tranches d'âge extrêmes (bébés et personnes âgées) conduit à un biais structurel que le modèle ne parvient pas à corriger malgré les tentatives d'amélioration décrites précédemment.

Par ailleurs, la prédiction de l'âge reste un problème intrinsèquement difficile. Le vieillissement facial varie considérablement d'un individu à l'autre en fonction de facteurs génétiques, environnementaux et ethniques. Cette variabilité naturelle constitue une source d'incertitude irréductible que tout modèle, quelle que soit sa complexité, ne peut éliminer complètement.

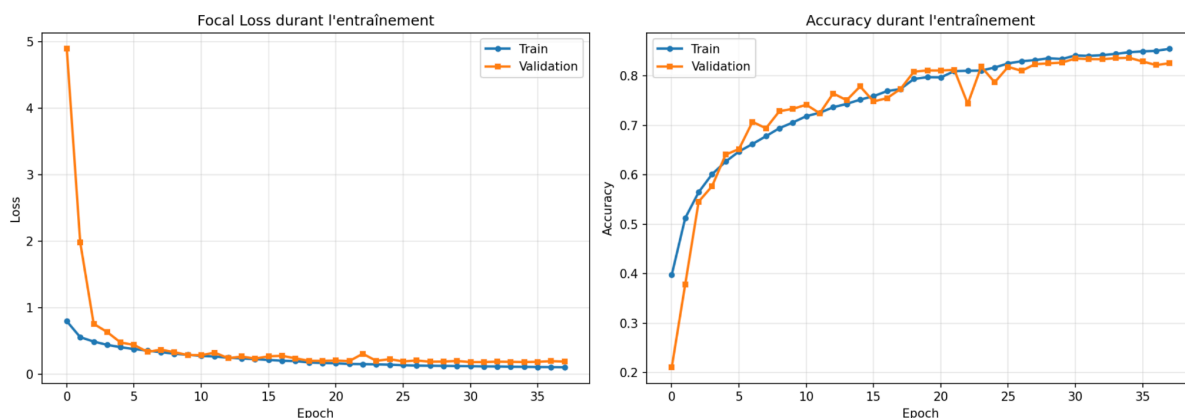
Enfin, la résolution des images (128×128 pixels) impose un compromis : elle est suffisante pour capturer les structures faciales générales, mais peut masquer des détails fins (rides, texture de la peau) potentiellement discriminants pour distinguer les âges proches.

Modèle pour l'ethnicité

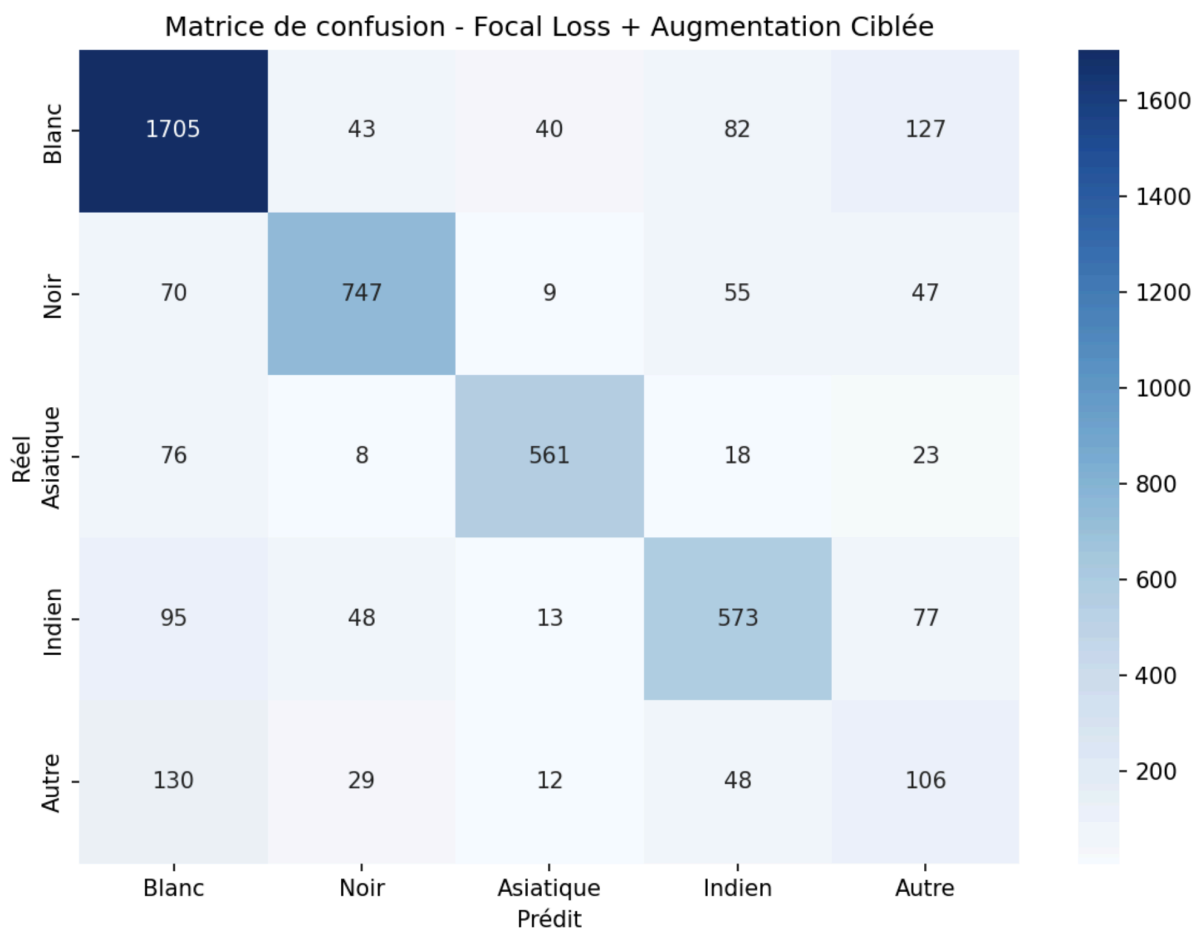
La prédiction de l'ethnie a été notre plus grand défi technique en raison du fort déséquilibre du dataset UTKFace, où les visages de catégorie "White" sont sur-représentés.

Plutôt que de simplement "punir" le modèle lorsqu'il se trompait sur les minorités, nous avons préféré lui apprendre mieux en utilisant une Augmentation de Données Ciblée. Nous avons généré artificiellement des milliers de variations (rotations, zooms) pour les groupes moins représentés (Indiens, Autres), afin de rééquilibrer la balance dès le départ. Sur certaines versions, nous avons même entraîné le modèle sur des images en niveaux de gris. Ce choix peut sembler surprenant, mais il force techniquement le réseau à se focaliser sur la morphologie et la structure du visage plutôt que sur la carnation de la peau, qui est souvent trompeuse selon l'éclairage. En combinant ces efforts avec une régularisation stricte (Dropout et L2), nous avons réussi à stabiliser les résultats au-delà de 80% de précision, offrant ainsi un modèle robuste et plus équitable.

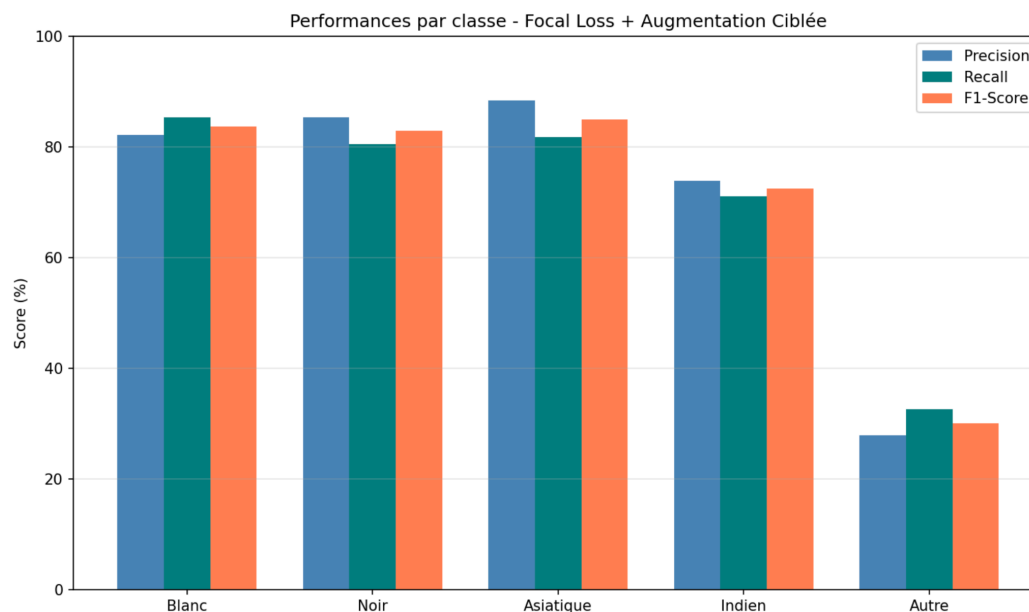
Nous avons fait évoluer notre architecture à travers plusieurs versions successives. Le modèle de base, un CNN simple à trois blocs convolutionnels avec pondération des classes (`class_weight`), atteint 74.67%. L'ajout d'augmentation de données classique (zoom, rotation) n'améliore pas significativement les résultats car elle ne résout pas le problème fondamental du déséquilibre des classes. Nous avons ensuite introduit des techniques architecturales plus avancées : les skip connections, inspirées de ResNet, qui créent des raccourcis permettant à l'information de traverser le réseau sans se dégrader au fil des couches, ainsi que les blocs SE-Net (Squeeze-and-Excitation), qui agissent comme un mécanisme d'attention en apprenant automatiquement quels filtres sont les plus importants pour la décision, et les convolutions séparables qui réduisent le nombre de paramètres sans perdre en capacité de détection. Ces améliorations portent l'accuracy à 74.27%. C'est finalement la combinaison de la Focal Loss et de l'augmentation ciblée qui permet le saut le plus significatif à 78.38%.



L'augmentation ciblée génère pour chaque classe sous-représentée des images supplémentaires par transformations variées (rotations de plus ou moins 15 degrés, zooms de plus ou moins 10%, flip horizontal, translations) jusqu'à atteindre exactement 8 081 images par classe, soit 40 405 images au total contre 18 966 à l'origine. Ce choix évite la double pénalisation que provoquerait la combinaison du `class_weight` avec l'augmentation. La Focal Loss, avec un paramètre gamma fixé à 3.0, complète cette approche en réduisant fortement la contribution des exemples bien classifiés et en forçant le modèle à se concentrer sur les cas difficiles, typiquement les visages ambigus entre deux ethnies.



Les résultats montrent des performances contrastées selon les classes : les catégories Blanc, Noir et Asiatique affichent des F1-scores supérieurs à 83%, démontrant que le modèle a appris des caractéristiques morphologiques discriminantes. La classe Indien présente des résultats corrects mais inférieurs (F1 de 73%), probablement en raison de la proximité morphologique avec d'autres groupes. La classe Autre reste le point faible (F1 de 30%), ce qui s'explique par la nature même de cette catégorie qui regroupe des visages sans étiquette ethnique claire.



Pour visualiser les zones du visage exploitées par le modèle, nous avons généré des heatmaps Grad-CAM sur des images du jeu de test. Ces visualisations confirment que le réseau se concentre sur les zones pertinentes du visage (contour, yeux, nez) plutôt que sur des artefacts de l'image. Lorsque le modèle est confiant (plus de 90%), les activations sont concentrées sur des traits morphologiques précis. En revanche, lorsqu'il se trompe, les activations sont dispersées, traduisant une incertitude dans l'identification des caractéristiques discriminantes.



b. Le modèle multi-tâche

Le modèle multi-tâche doit être capable d'identifier les trois caractéristiques simultanément. Pour ce modèle nous utilisons les images en couleurs car elle donne des informations essentielles pour l'ethnicité. Malgré que le coût de calcul soit plus élevé, ce choix améliore les performances globales du modèle multi-tâches.

Pour ce genre de modèle nous utilisons un réseau de neurones convolutif (Convolutional Neural Network CNN). Ce modèle est structuré par une première couche de base qui permet d'extraire les patterns utiles, suivie de trois branches de sorties (une branche pour chaque tâche). Nous effectuons également une étape de data augmentation en amont comme pour les autres modèles afin d'améliorer la capacité de généralisation du modèle en simulant des variations sur les images (rotation, zoom, etc.). Puis, l'image est traitée par plusieurs blocs convolutionnels successifs, chacun composé :

- **d'une couche Conv2D** : pour extraire les motifs.
- **d'une couche de normalisation (Batch Normalization)** : Batch Normalization normalise les activations pour stabiliser et accélérer l'apprentissage en réduisant le phénomène de gradient instable. Elle est utilisée après la couche Conv2D car cette dernière peut produire des valeurs très grandes et dispersées.
- **d'une couche d'activation ReLU** : pour ne garder que les signaux utiles (les valeurs négatives deviennent 0 et les valeurs positives restent tel quel).
- **d'une couche de Max Pooling** : afin de réduire la dimension des images tout en gardant les informations importantes.

Nous avons débuté avec une architecture de base, composée de trois blocs convolutifs ayant respectivement 32, 64 et 128 filtres chacun. Nous avons fait ce choix pour débiter car il permet une progression classique et l'extraction des caractéristiques de plus en plus complexes au fur et à mesure que la profondeur du réseau augmente. De plus, c'est une architecture assez classique qui permet d'avoir un compromis raisonnable entre le coût et la performance. Ces blocs sont suivis d'un bloc de global average pooling, nous avons choisi le Global Average Pooling au lieu d'une couche Flatten, car il réduit le nombre de paramètres du modèle en remplaçant une vectorisation complète des cartes de caractéristiques (transformation des valeurs d'un tenseur en un seul vecteur 1D, sans perdre aucune information) par une moyenne globale de celles-ci. Cela limite le sur-apprentissage. De plus, le Global Average Pooling conserve l'information globale contenue dans chaque carte de caractéristiques, ce qui est suffisant pour des tâches de classification ou de régression, tout en améliorant la robustesse du modèle et en réduisant le coût computationnel.

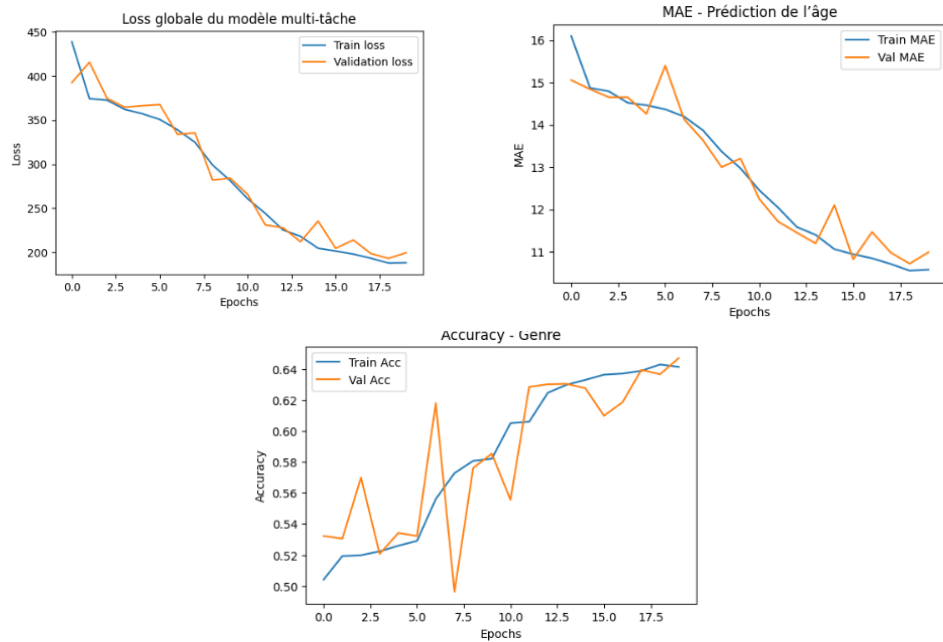
Nous ajoutons ensuite une couche dense de 128 neurones. Cette couche oblige le réseau à condenser les caractéristiques extraites en une représentation compacte avant la séparation en trois branches distinctes. Enfin, trois têtes de sortie pour l'âge (régression), le genre (binaire) et l'ethnicité (5 classes). Nous utilisons l'optimiseur Adam, qui permet au modèle d'adapter ses poids de manière dynamique lors de l'apprentissage et d'obtenir une convergence plus rapide. La fonction de perte totale est une combinaison des pertes associées à chaque tâche, ce qui permet d'optimiser simultanément les trois objectifs. Dans notre cas les pertes ne sont pas encore pondérées, ce qui signifie qu'on leur accorde la même importance. Ainsi, les trois branches de sortie partagent une

base d'apprentissage commune issue des couches convolutionnelles, tout en ayant des paramètres adaptés à chaque tâche dans les couches finales.

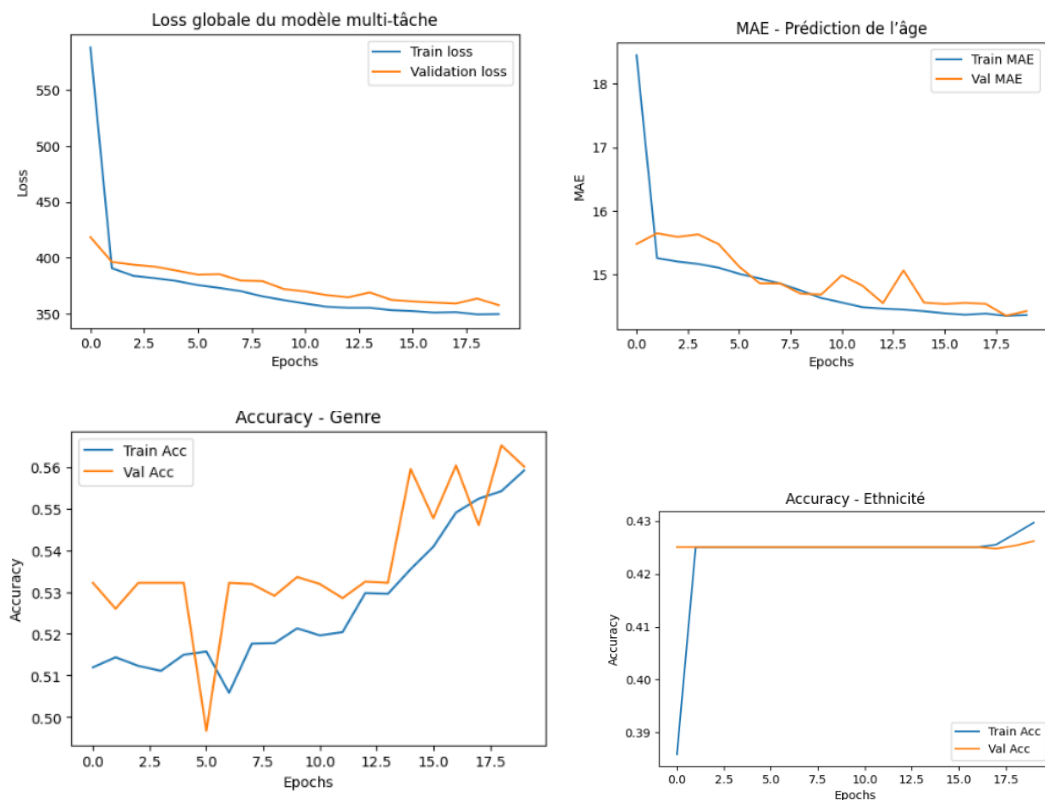
```
# -----  
# 1. Entrée du modèle  
# -----  
input_img = layers.Input(shape=(128, 128, 1))  
  
# -----  
# 2. Bloc convolutionnel 1  
# -----  
x = layers.Conv2D(32, (3, 3), padding='same')(input_img)  
x = layers.ReLU()(x)  
x = layers.MaxPooling2D((2, 2))(x)  
  
# -----  
# 3. Bloc convolutionnel 2  
# -----  
x = layers.Conv2D(64, (3, 3), padding='same')(x)  
x = layers.ReLU()(x)  
x = layers.MaxPooling2D((2, 2))(x)  
  
# -----  
# 4. Bloc convolutionnel 3  
# -----  
x = layers.Conv2D(128, (3, 3), padding='same')(x)  
x = layers.ReLU()(x)  
  
# -----  
# 5. Global Pooling  
# -----  
x = layers.GlobalAveragePooling2D()(x)  
  
# -----  
# 6. Couche Dense partagée  
# -----  
x = layers.Dense(128, activation='relu')(x)  
  
# -----  
# 7. Sorties multi-tâches  
# -----  
age_output = layers.Dense(1, activation='linear', name='age')(x)  
gender_output = layers.Dense(1, activation='sigmoid', name='gender')(x)  
ethnicity_output = layers.Dense(5, activation='softmax', name='ethnicity')(x)  
  
# -----  
# 8. Modèle final  
# -----  
model = models.Model(  
    inputs=input_img,  
    outputs=[age_output, gender_output, ethnicity_output]  
)
```

Nous définissons ensuite **les règles d'apprentissage du modèle**. Pour la fonction de perte (la loss) nous utilisons la **MSE** (adaptée pour les tâches de régression où la sortie est un nombre continu) pour l'âge, la **binary crossentropy** pour le genre et la **categorical cross entropy** pour l'ethnicité. Les performances du modèle sont évaluées avec une métrique différente et adaptée pour chaque apprentissage: la **MAE** pour l'âge et l'**accuracy** pour le genre et l'ethnie. La MAE donne directement l'erreur moyenne en années, ce qui est une donnée assez claire et intuitive pour évaluer la performance du modèle. L'accuracy quant à elle est adaptée aux tâches de classification car elle indique la proportion de prédictions correctes.

Une fois ces caractéristiques définies nous procédons à l'apprentissage en **20 epochs** dans un premier temps pour voir comment le modèle prédit les labels et en utilisant des lot (**batches**) de taille 32 pour que le modèle mettent à jour ses poids toutes les 32 images traitées et ainsi avoir un apprentissage plus stable et efficace.



Ces métriques pourraient être plus stables donc nous ajustons le taux d'apprentissage (learning rate) à $1e-4$ pour améliorer la stabilité sans changer l'architecture. Cela permet une meilleure optimisation des poids car les poids seront mis à jour plus progressivement et ainsi on aura une convergence plus douce et fiable.



Les métriques montrent une convergence stable sans surapprentissage, notamment avec la proximité des courbes d'entraînement et de validation. La prédiction de l'âge s'améliore progressivement, en revanche, les performances sur le genre restent encore instables et limitées, ce

qui montre une difficulté à extraire des caractéristiques discriminantes. Le point critique concerne la prédiction de l'ethnicité, dont les performances stagnent malgré l'entraînement, ce qui s'explique par le déséquilibre des classes représentées dans la dataset et par une fonction de perte dominante. Cela nous indique que nous devons ajuster les pondérations de loss et potentiellement approfondir le réseau de neurones. Nous faisons ainsi évoluer notre modèle à travers plusieurs versions jusqu'à atteindre le modèle le plus performant possible dans les limites du datasets.

Finalement, nous arrivons au modèle suivant :

```
#-----  
# Bloc 1  
#-----  
x = layers.Conv2D(32, (3, 3), padding='same')(x)  
x = layers.BatchNormalization()(x)  
x = layers.Activation('relu')(x)  
x = layers.MaxPooling2D((2, 2))(x)  
  
#-----  
# Bloc 2  
#-----  
x = layers.Conv2D(64, (3, 3), padding='same')(x)  
x = layers.BatchNormalization()(x)  
x = layers.Activation('relu')(x)  
x = layers.MaxPooling2D((2, 2))(x)  
  
#-----  
# Bloc 3  
#-----  
x = layers.Conv2D(128, (3, 3), padding='same')(x)  
x = layers.BatchNormalization()(x)  
x = layers.Activation('relu')(x)  
x = layers.MaxPooling2D((2, 2))(x)  
  
#-----  
# Bloc 4  
#-----  
x = layers.Conv2D(256, (3, 3), padding='same')(x)  
x = layers.BatchNormalization()(x)  
x = layers.Activation('relu')(x)  
x = layers.MaxPooling2D((2, 2))(x)  
  
#-----  
# Bloc 5  
#-----  
x = layers.Conv2D(512, (3, 3), padding='same')(x)  
x = layers.BatchNormalization()(x)  
x = layers.Activation('relu')(x)  
x = layers.MaxPooling2D((2, 2))(x)  
  
#-----  
# Global  
#-----  
x = layers.GlobalAveragePooling2D()(x)  
  
# Branche Âge  
age_b = layers.Dense(256, activation='relu')(x)  
age_b = layers.Dense(128, activation='relu')(age_b)  
age_out = layers.Dense(1, activation='sigmoid', name='age')(age_b)  
  
# Branche Genre  
gen_b = layers.Dense(128, activation='relu')(x)  
gen_b = layers.Dropout(0.3)(gen_b)  
gen_out = layers.Dense(2, activation='softmax', name='gender')(gen_b)  
  
# Branche Ethnicité  
eth_b = layers.Dense(256, activation='relu')(x)  
eth_b = layers.Dense(128, activation='relu')(eth_b)  
eth_b = layers.Dropout(0.4)(eth_b)  
eth_out = layers.Dense(5, activation='softmax', name='ethnicity')(eth_b)
```

Ce modèle est composé de manière suivante: Une **base convolutionnelle** composée de **cinq blocs** successifs (32, 64, 128, 256 et 512 filtres) avec une couche de **Batch Normalization**, d'une couche d'activations **ReLU** et une couche de **MaxPooling**. Nous avons toujours l'étape de data augmentation (rotations, zooms, symétries) directement intégrée au modèle. Nous avons gardé la structure multi-têtes avec une branche spécifique par tâche : une régression pour l'âge, et deux classifications pour le genre et l'ethnicité. Cette approche permet de partager une représentation commune tout en spécialisant les prédictions, ce qui est utile lorsque les tâches sont liées. Les fonctions de perte sont aussi adaptées à chaque tâche. Pour la fonction de perte de l'âge nous avons changé la MSE par la **Huber** pour limiter l'impact des valeurs trop incohérentes: le fonction Huber fait la même chose

que la MSE pour les petites erreurs ce qui permet un apprentissage précis et réduit l'impact des grandes erreurs. En effet, la MSE pénalise beaucoup les grandes erreurs (car elle met au carré les écarts) ce qui pose problème pour les erreurs d'âge car nous savons que c'est la tâche la plus difficile. C'est pourquoi l'utilisation de Huber évite que ces erreurs pénalisent trop l'apprentissage global du modèle. Pour le genre nous avons gardé la fonction **cross entropy**, et pour l'ethnie une **focal loss** qui modifie la cross entropy en donnant plus de poids aux exemples compliqués à classer et moins aux exemples simples à prédire, ce qui permet d'améliorer les performances sur les classes minoritaires. Les résultats de ces fonctions sont additionnés avec un système de **pondération** qui donne plus d'importance à la tâche de prédiction de l'ethnie (pondérée à 2 contre 1 pour les deux autres tâches) car c'est la plus impactée par le déséquilibre des classes dans le dataset. L'**optimiseur Adam** est utilisé pour sa capacité à ajuster dynamiquement le taux d'apprentissage et à converger efficacement. Enfin, des techniques de régularisation comme le **Dropout**, l'**Early Stopping** et la réduction adaptative du learning rate (**ReduceLROnPlateau**) sont mises en place pour limiter le surapprentissage et améliorer les performances sur des données non vues. Le Dropout consiste à désactiver aléatoirement une partie de neurones pendant l'entraînement afin d'éviter que le modèle ne deviennent trop dépendants de certains d'entre eux. L'early Stopping permet d'arrêter automatiquement l'entraînement au moment où les performances ne s'améliorent plus et ainsi d'éviter le sur apprentissage. Le reduceLROnPlateau diminue automatiquement le taux d'apprentissage lorsque la performance stagne ce qui permet d'avoir un apprentissage plus fin (En réduisant la taille des mises à jour, le modèle peut mieux se stabiliser et se rapprocher d'un minimum optimal de la fonction de perte.).



Ces metrics ne sont pas les plus stables mais il est difficile d'obtenir de meilleur résultat pour ce type de modèle avec UTKFace. En effet, avec un modèle entraîné de zéro sur des données variables en termes d'éclairage, d'angle, de qualité etc et sur un dataset déséquilibré rendent l'apprentissage compliqué. De plus, sans utiliser de transfer learning, le modèle est limité ce qui explique que le modèle, même avec une architecture solide, peut présenter des variations de performances.

Les performances du modèle sont moins bonnes lorsqu'on l'utilise dans notre application. Cela s'explique par les différences entre les photos d'entraînement (UTKFace) et les photos réelles. Les photos utilisées dans l'application peuvent varier en éclairage, cadrage, qualité.

c. Le modèle par transfert de connaissance

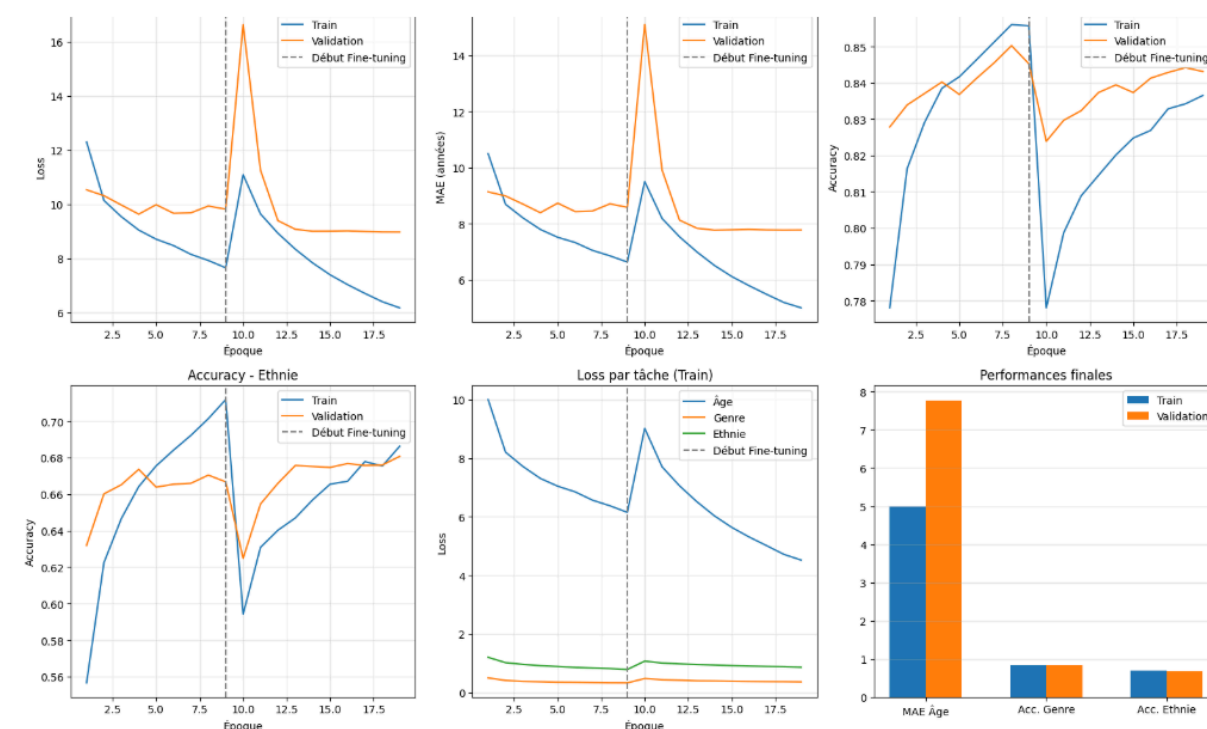
Pour obtenir une application fluide et réactive sur smartphone, nous avons privilégié le Transfer Learning avec l'architecture MobileNetV2. Ce modèle, pré-entraîné sur des millions d'images (ImageNet), possède une connaissance approfondie des formes et des textures, ce qui nous a permis de gagner un temps précieux en entraînement.

Nous avons choisi MobileNetV2 spécifiquement pour sa légèreté et son efficacité sur les processeurs mobiles. L'entraînement s'est fait de manière progressive :

Phase de "Head-only" : Nous avons d'abord entraîné uniquement nos nouvelles couches de prédiction (les "têtes" pour l'âge, le genre et l'ethnie) tout en laissant le modèle de base figé. Cela permet de stabiliser les prédictions sans détruire les connaissances pré-existantes du modèle.

Phase de "Fine-tuning" : Nous avons ensuite dégelé les dernières couches de MobileNetV2 pour les ajuster très finement aux traits spécifiques des visages du dataset UTKFace. C'est cette étape, bien que délicate, qui nous a permis d'affiner nos prédictions.

Modèle 01 : MobileNetV2



Analyse des performances de l'entraînement (Modèle MobileNetV2)

1. Analyse de la fonction de perte (Loss totale)

La courbe de Loss totale montre une décroissance rapide et régulière durant la Phase 1 (entraînement des têtes de lecture). Lors de la transition vers la Phase 2 (Fine-tuning, marqué par la ligne pointillée grise), on observe un pic brutal de la perte de validation. Ce phénomène est caractéristique d'un réajustement des poids pré-entraînés de MobileNetV2. Après ce choc initial, la perte se stabilise à un niveau inférieur à celui de la phase 1, prouvant l'efficacité de l'ajustement fin des couches supérieures.

2. Prédiction de l'âge (Régression)

Le graphique MAE - Âge indique que l'erreur moyenne absolue descend aux alentours de **7,7 ans** sur le jeu de validation.

Observation notable : C'est sur cette métrique que le Fine-tuning a eu l'impact le plus fort, bien qu'instable. L'écart entre la courbe d'entraînement (bleue) et de validation (orange) reste contenu, ce qui suggère que le modèle généralise convenablement sans entrer dans un sur-apprentissage (overfitting) massif.

3. Classification du Genre et de l'Ethnie

Genre (Accuracy) : Le modèle atteint un plateau rapide à environ **84% de précision**. On note cependant que le Fine-tuning n'a pas permis de gain significatif sur cette tâche. Cela peut s'expliquer par un biais de classification où le modèle, pour minimiser la perte globale, a tendance à privilégier la classe majoritaire (biais statistique du dataset UTKFace).

Ethnie (Accuracy) : La précision culmine à **68%**. Compte tenu de la complexité de cette tâche (5 classes distinctes) et de la subtilité des traits morphologiques, ce résultat est très satisfaisant. La courbe de validation suit de près celle d'entraînement, confirmant une bonne robustesse.

4. Répartition de la perte par tâche

Le graphique Loss par tâche révèle une hiérarchie claire : la tâche de l'âge génère une perte numérique beaucoup plus importante que les tâches de classification (genre et ethnie). Ce déséquilibre explique pourquoi le modèle "néglige" parfois la précision du genre : l'optimiseur mathématique se concentre prioritairement sur la réduction de l'erreur d'âge, car elle pèse plus lourd dans le calcul du gradient global.

Synthèse des performances finales

Le graphique à barres récapitulatif confirme l'équilibre global du modèle :

- **Âge** : MAE ~7.8 ans.
- **Genre** : Précision ~84%.
- **Ethnie** : Précision ~68%.

Conclusion de l'analyse : Le modèle est fonctionnel et performant. Pour les itérations futures, une normalisation des étiquettes d'âge ou une pondération plus agressive de la loss du genre (via `loss_weights`) permettrait de réduire les biais de classification observés et d'équilibrer davantage l'apprentissage entre les trois branches.

Modèle 02 : EfficientNetB0

Parallèlement à nos tests sur MobileNetV2, nous avons développé une architecture basée sur EfficientNetB0. Ce modèle représente l'état de l'art en matière d'efficacité : il utilise un mécanisme appelé Compound Scaling qui équilibre la profondeur et la largeur du réseau pour obtenir une précision maximale.

Une architecture sophistiquée :

Ce modèle se distingue par plusieurs innovations techniques que nous avons intégrées :

Mécanisme d'Attention (SE-Blocks) : EfficientNet inclut nativement des blocs Squeeze-and-Excitation. Cela permet au réseau de “décider” quels traits du visage sont les plus importants (par exemple, ignorer le fond pour se concentrer sur les rides ou la forme des yeux).

Focal Loss (Gestion des minorités) : Pour l'éthnicité, nous avons remplacé la fonction de perte classique par une Focal Loss. Cette fonction mathématique “force” le modèle à se concentrer sur les visages les plus difficiles à classer (les classes minoritaires du dataset UTKFace) en réduisant l'importance des exemples trop faciles.

Mixed Precision (Calcul accéléré) : Nous avons utilisé la précision mixte (float16). Cela permet au modèle de s'entraîner beaucoup plus vite sur GPU tout en utilisant moins de mémoire vive.

Stratégie d'apprentissage par paliers (3 Phases)

Pour tirer le meilleur de ce modèle, nous avons mis en place un processus de dégel progressif (unfreezing) en trois étapes :

Phase 1 (Échauffement) : Entraînement des têtes de lecture uniquement.

Phase 2 (Ajustement partiel) : Libération des 10 dernières couches du modèle pour commencer à adapter les filtres complexes.

Phase 3 (Ajustement profond) : Libération des 50 dernières couches pour une spécialisation totale sur la morphologie humaine.

4. L'application Android

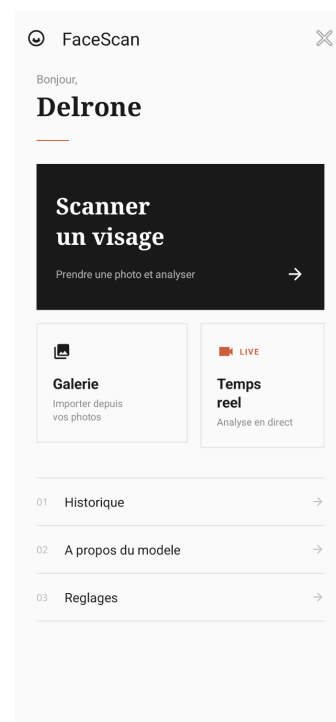
L'application mobile est la partie visible de notre projet. Elle sert d'interface entre nos modèles d'intelligence artificielle et l'utilisateur final. Nous avons fait le choix d'un développement natif en Java afin d'exploiter au mieux les capacités de calcul des smartphones modernes, un point essentiel pour exécuter des modèles d'IA de manière fluide.

a. Architecture technique et gestion des modèles

Pour que l'application reste rapide et agréable à utiliser, nous avons séparé l'interface graphique de la logique de calcul. Le coeur du système est le FaceAnalyzer, un module que nous avons conçu pour piloter l'ensemble de la chaîne de prédiction : détection du visage, prétraitement de l'image et inférence via les modèles

L'intégration de nos modèles repose sur TensorFlow Lite, la version allégée de TensorFlow conçue pour les appareils mobiles. Ce choix est stratégique car il permet à l'application de fonctionner totalement hors-ligne : les photos ne quittent jamais le téléphone, ce qui garantit la protection de la vie privée de l'utilisateur. Cinq fichiers TFLite sont embarqués directement dans l'application : un modèle multi-tâches, un modèle par transfert learning, et trois modèles spécialisés pour l'âge, le genre et l'ethnicité. L'utilisateur peut choisir entre ces trois stratégies via un menu de réglages, et ce choix est sauvegardé localement pour être appliqué automatiquement à chaque analyse.

Pour éviter que l'écran ne se fige pendant une analyse, l'ensemble du traitement lourd (détection de visage, conversion d'image, inférence) tourne sur des threads séparés du thread principal. Des buffers mémoire réutilisables évitent les allocations répétées qui ralentiraient l'analyse, et un mécanisme de protection empêche les crashes lorsque l'utilisateur quitte un écran pendant qu'une analyse est encore en cours.



b. Le traitement de l'image : de la caméra à l'IA

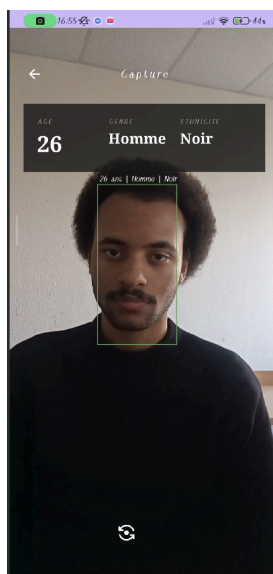
Transformer une image brute provenant de la caméra en une donnée exploitable par l'IA a été l'un de nos plus gros défis techniques. Le pipeline de traitement se décompose en plusieurs étapes.

La première étape est la détection du visage. Nous utilisons Google ML Kit Face Detection pour identifier et recadrer automatiquement le visage principal dans l'image. Le système détecte le plus grand visage, extrait sa zone avec une marge de 20% autour pour conserver le contour, puis croppe cette zone. Si aucun visage n'est détecté, l'image originale est utilisée en fallback. Cette étape améliore significativement la précision des prédictions car le modèle reçoit uniquement le visage, sans le fond, les vêtements ou d'autres éléments parasites.

La deuxième étape est le prétraitement. Le visage recadré est redimensionné en 128x128 pixels, puis normalisé selon le modèle sélectionné. Les modèles spécialisés et multi-tâches reçoivent une image en niveaux de gris normalisée entre 0 et 1, tandis que le modèle par transfert learning attend une image RGB normalisée entre -1 et 1, le format attendu par MobileNetV2. L'application détecte même automatiquement le nombre de canaux attendu par le modèle multi-tâches pour choisir le bon prétraitement.

Un point technique important concerne l'identification des sorties du modèle TFLite. L'ordre des tenseurs de sortie peut varier d'un modèle à l'autre après la conversion. L'application utilise donc une stratégie de détection en trois niveaux : d'abord par le nom du tenseur, puis par le suffixe numérique, et enfin par la forme du tenseur en fallback (un tenseur de taille 5 correspond à l'ethnicité, de taille 2 au genre, de taille 1 à l'âge). Certaines photos prises par le téléphone sont sauvegardées avec une orientation incorrecte dans les métadonnées EXIF. L'application corrige automatiquement cette rotation avant l'analyse pour éviter d'envoyer une image tournée au modèle.

c. Fonctionnalités et expérience utilisateur



L'application propose deux modes d'analyse principaux.

Le mode capture permet de prendre une photo ou d'en importer une depuis la galerie. L'image passe par la détection de visage puis par le modèle sélectionné. L'écran de résultats affiche le visage recadré accompagné de l'âge, du genre et de l'ethnicité avec leurs scores de confiance sous forme de barres de progression. L'utilisateur peut sauvegarder le résultat dans Firebase Firestore ou le partager.

Le mode temps réel est la fonctionnalité la plus complexe techniquement. Il utilise le composant ImageAnalysis de CameraX pour analyser les images de la caméra en continu. La caméra produit des images au format YUV420, le format natif des capteurs Android, à une résolution de 640x480. Chaque image est convertie en Bitmap par une reconstruction manuelle du buffer NV21 à partir des trois plans Y, U et V. Un mécanisme de throttle limite

l'analyse à une frame toutes les 200 millisecondes, soit environ 5 images par seconde. Les images intermédiaires sont ignorées grâce à la stratégie KEEP_ONLY_LATEST de CameraX, ce qui évite toute accumulation en file d'attente. Un rectangle vert est dessiné en temps réel autour du visage détecté avec un label affichant les prédictions.

Le mode spécialisé exécute trois modèles séparés en séquence pour l'âge, le genre et l'ethnicité. Cette approche n'est pas significativement plus lente que le modèle multi-tâches car chaque modèle spécialisé est léger (environ 500 Ko) et son inférence ne prend que quelques millisecondes. Le goulot d'étranglement se situe dans la détection de visage et la conversion des images, qui représentent la majorité du temps de traitement. Trois inférences de 3 à 5 millisecondes restent négligeables face aux 50 à 100 millisecondes du reste du pipeline.

L'application intègre également un système d'authentification via Firebase Authentication, un historique des prédictions stocké dans Firestore avec possibilité de suppression individuelle ou globale, et un écran d'information détaillant le projet, les modèles et les métriques utilisées.

